Beautiful Soup 4

A practical guide with a worked example page

Prepared for Krish · bs4 4.15.0, soupsieve 3.0.3, Python 3 · 12 October 2026

How to use this guide

1. Read Section 2. It shows the dummy page `bookstore.html`. All examples use this page.
2. Read each example box. The upper part is the Python code. The lower part is the real output.
3. Use the *Targets* label. It gives the line numbers in the HTML listing (Listing 1, page 3) that the code reads.
4. Run the code yourself. Save the HTML as `bookstore.html` next to your script.
5. Use the cheat sheet (Section 13) as a quick reference.

Starting state of every example

Each example starts from a fresh parse of the page with this code. The examples do not repeat it.

```
from bs4 import BeautifulSoup, Comment
import re

with open("bookstore.html", encoding="utf-8") as f:
    soup = BeautifulSoup(f, "html.parser")
```

All outputs come from a real run with bs4 4.15.0 and the `html.parser` parser. API facts follow the official documentation: https://www.crummy.com/software/BeautifulSoup/bs4/doc/.

Contents

1 Install and set up . . . 1
2 The dummy website . . . 2
3 Make the soup . . . 5
4 The four object types . . . 5
5 Navigate the tree . . . 6
6 Search the tree . . . 10
7 CSS selectors: select() and select_one() . . . 13
8 Get the data . . . 14
9 Modify the tree (brief) . . . 15
10 Output . . . 16
11 A short scraping workflow . . . 17
12 Common pitfalls . . . 19
13 Cheat sheet . . . 21
A Glossary . . . 22

1 Install and set up

Beautiful Soup (bs4) reads HTML or XML. It builds a tree of Python objects from the markup. You search and read that tree.

1.1 Install the packages

1. Open a terminal (or a notebook cell with a leading `!`).
2. Install the library. The package name is `beautifulsoup4`. The import name is `bs4`.
3. Install the optional parsers `lxml` and `html5lib`.
4. Install `requests` and `pandas` the same way, for the workflow in Section 11.

```
pip install beautifulsoup4 lxml html5lib
```

Output (last line, fresh virtual environment) Targets: No page needed

```
Successfully installed beautifulsoup4-4.15.0 html5lib-1.1 lxml-6.1.3 six-1.17.0 soupsieve-3.0.3
↪  typing-extensions-4.16.0 webencodings-0.6.1
```

Do not install `BeautifulSoup`

The package `BeautifulSoup` (without the 4) is Beautiful Soup 3, the old release. Install `beautifulsoup4`. pip also installs `soupsieve`, which runs the CSS selectors.

Verify the install:

```
import bs4, lxml, html5lib, soupsieve
print("beautifulsoup4", bs4.__version__)
print("soupsieve", soupsieve.__version__)
```

Output Targets: No page needed

```
beautifulsoup4 4.15.0
soupsieve 3.0.3
```

1.2 Choose a parser

A parser turns the markup text into a tree. bs4 supports these parsers (table from the official documentation):

Parser	Usage	Advantages	Disadvantages
Python's html.parser	`BeautifulSoup(m, "html.parser")`	Included with Python. Decent speed.	Not as fast as lxml. Less lenient than html5lib.
lxml (HTML)	`BeautifulSoup(m, "lxml")`	Very fast.	External C dependency.
lxml (XML)	`BeautifulSoup(m, "xml")`	Very fast. Only supported XML parser.	External C dependency.
html5lib	`BeautifulSoup(m, "html5lib")`	Extremely lenient. Parses like a web browser. Creates valid HTML5.	Very slow. External Python dependency.

The parsers give the same tree for a valid page. They give different trees for broken HTML:

```
markup = "<a></p>"
for parser in ["html.parser", "lxml", "html5lib"]:
    print(parser, "->", BeautifulSoup(markup, parser))
```

Output — Targets: Not the dummy page (small invalid snippet from the docs)

```
html.parser -> <a></a>
lxml -> <html><body><a></a></body></html>
html5lib -> <html><head></head><body><a><p></p></a></body></html>
```

Rule

Always name the parser in the constructor. Then your script gives the same tree on every computer. This guide uses `"html.parser"`, because it tracks line numbers (`tag.sourceline`). The lxml parsers do not track line numbers.

2 The dummy website

The page `bookstore.html` is a small shop page. It has a head, a navigation bar, a header, a list of four books, a stock table, and a footer with an HTML comment. A separate file `style.css` gives the look. bs4 does not read the CSS file.

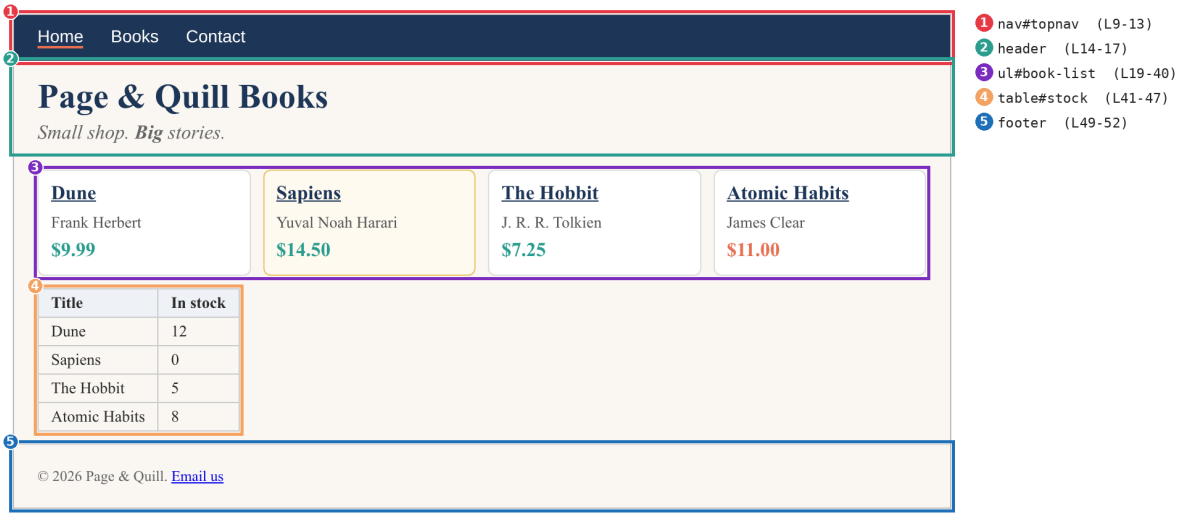


Figure 1. The page in a headless Chromium browser. Numbered boxes mark the main regions and their lines in Listing 1.

Region	Lines	Hooks for selection
`head`	3–7	`meta charset`, `title`, `link rel="stylesheet"`
`nav#topnav`	9–13	three `a.nav-link`; the first also has class `active`
`header`	14–17	`h1`; `p.tagline` with mixed content (text + `b`)
`ul#book-list`	19–40	four `li.book` with `id` b1–b4 and `data-genre`; `h2.title > a`, `span.author`, `span.price`; b2 is also `.bestseller`; b4's price is also `.sale`
`table#stock`	41–47	one header row (`th`) and four data rows (`td`); no `tbody` in the source
`footer`	49–52	an HTML comment; a `p` with a literal © and a `mailto:` link

Listing 1. bookstore.html (line numbers on the left)

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <title>Page &amp; Quill Books</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <nav id="topnav">
    <a href="/" class="nav-link active">Home</a>
    <a href="/books" class="nav-link">Books</a>
    <a href="/contact" class="nav-link">Contact</a>
  </nav>
  <header>
    <h1>Page &amp; Quill Books</h1>
    <p class="tagline">Small shop. <b>Big</b> stories.</p>
  </header>
  <main>
    <ul id="book-list">
      <li class="book" id="b1" data-genre="fiction">
        <h2 class="title"><a href="/books/dune">Dune</a></h2>
        <span class="author">Frank Herbert</span>
        <span class="price">$9.99</span>
      </li>
      <li class="book bestseller" id="b2" data-genre="history">
        <h2 class="title"><a href="/books/sapiens">Sapiens</a></h2>
        <span class="author">Yuval Noah Harari</span>
        <span class="price">$14.50</span>
      </li>
      <li class="book" id="b3" data-genre="fiction">
        <h2 class="title"><a href="/books/hobbit">The Hobbit</a></h2>
        <span class="author">J. R. R. Tolkien</span>
        <span class="price">$7.25</span>
      </li>
      <li class="book" id="b4" data-genre="self-help">
        <h2 class="title"><a href="/books/atomic-habits">Atomic Habits</a></h2>
        <span class="author">James Clear</span>
        <span class="price sale">$11.00</span>
      </li>
    </ul>
    <table id="stock">
      <tr><th>Title</th><th>In stock</th></tr>
      <tr><td>Dune</td><td>12</td></tr>
      <tr><td>Sapiens</td><td>0</td></tr>
      <tr><td>The Hobbit</td><td>5</td></tr>
      <tr><td>Atomic Habits</td><td>8</td></tr>
    </table>
  </main>
  <footer>
    <!-- Prices updated 2026-10-01 -->
    <p>© 2026 Page &amp; Quill. <a href="mailto:hello@pagequill.example">Email us</a></p>
  </footer>
</body>
</html>
```

2.1 The parse tree

Figure 2 shows the tree that `html.parser` builds. Each box is one Tag. The label gives the tag name, then the `id` (#) or first class (.), then the source line. The figure leaves out the doctype and the text strings. It keeps the comment.

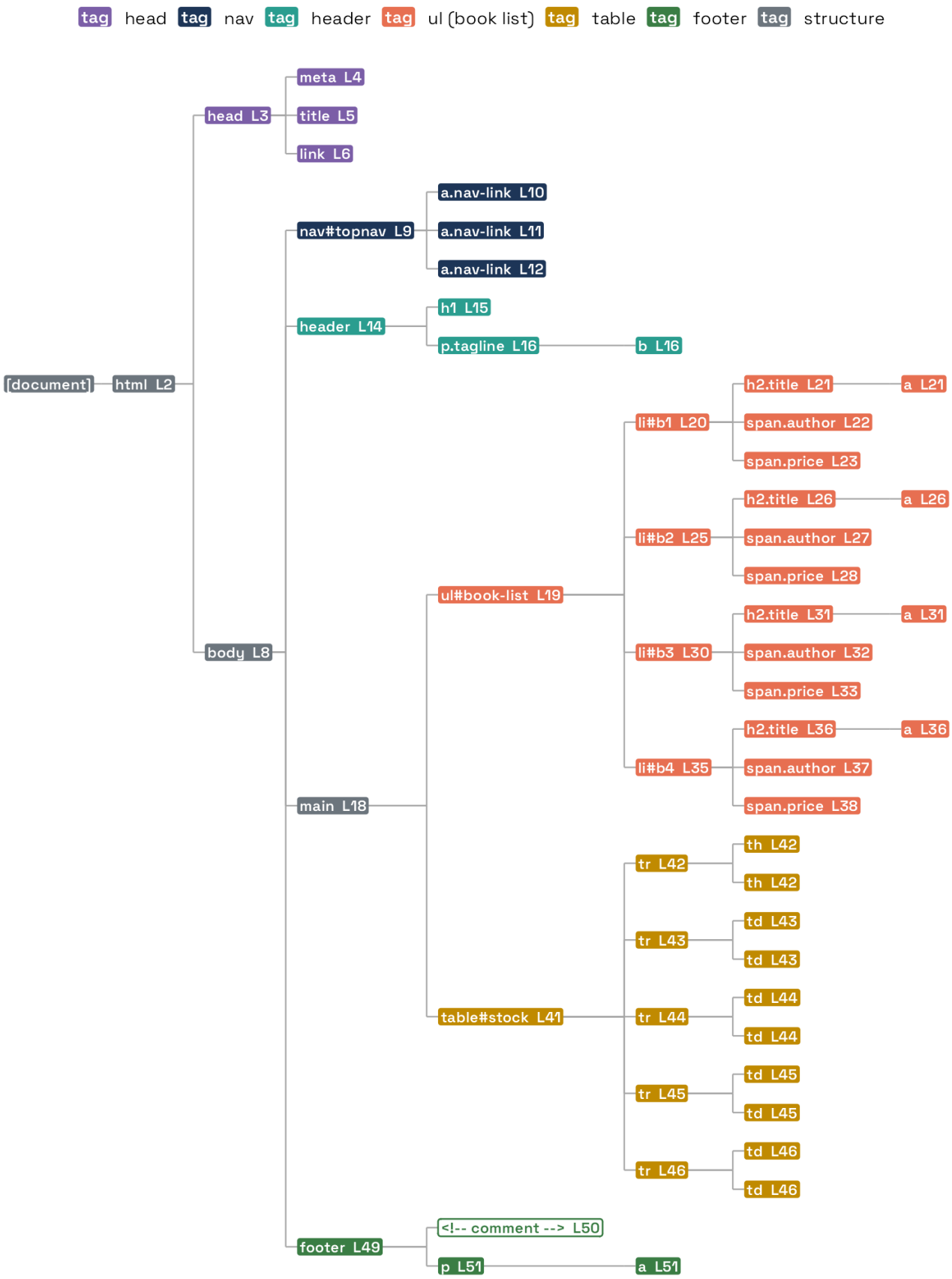


Figure 2. Parse tree of `bookstore.html` (tags only, plus the comment). Generated with R (ggplot2) from the real bs4 tree.

3 Make the soup

1. Import the class: `from bs4 import BeautifulSoup`.
2. Pass the markup as the first argument. Use an open file or a string.
3. Pass the parser name as the second argument.

```
from bs4 import BeautifulSoup

with open("bookstore.html", encoding="utf-8") as f:
    soup = BeautifulSoup(f, "html.parser")

print(type(soup))
print(soup.title)
```

Output — Targets: HTML line 5

```
<class 'bs4.BeautifulSoup'>
<title>Page &amp;amp; Quill Books</title>
```

A string works the same way:

```
snippet = "<p class='tagline'>Small shop. <b>Big</b> stories.</p>"
mini = BeautifulSoup(snippet, "html.parser")
print(mini.b)
```

Output — Targets: A string copy of HTML line 16

```
<b>Big</b>
```

bs4 converts the document to Unicode. It converts HTML entities (`&`) to characters. `print(tag)` writes `&` back as `&`, because the output is HTML. `get_text()` gives the plain character.

```
print(soup.h1)
print(soup.footer.p.get_text())
```

Output — Targets: HTML lines 15, 51

```
<h1>Page &amp;amp; Quill Books</h1>
© 2026 Page & Quill. Email us
```

4 The four object types

The tree contains four main kinds of object.

Type	What it is	Example on the page
`Tag`	One HTML element. It has `.name` and `.attrs`.	`<li id="b1">` (line 20)
`NavigableString`	A piece of text inside a tag. It acts like a Python `str`.	`Dune` (line 21)
`BeautifulSoup`	The whole document. It acts like a Tag. Its `.name` is `[document]`.	`soup`
`Comment`	A special NavigableString for `<!-- ... -->`.	line 50

4.1 Tag: name and attributes

```
tag = soup.find("li")
print(type(tag))
print(tag.name)
print(tag.attrs)
```

Output — Targets: HTML lines 20--24

```
<class 'bs4.element.Tag'>
li
{'class': ['book'], 'id': 'b1', 'data-genre': 'fiction'}
```

`class` is a multi-valued attribute. bs4 always gives it as a list. `id` is a single value, so bs4 gives a string.

```
print(soup.find(id="b1")["class"])
print(soup.find(id="b2")["class"])
print(soup.find(id="b2")["id"])
```

Output Targets: HTML lines 20--24, 25--29

```
['book']
['book', 'bestseller']
b2
```

4.2 NavigableString

```
text = soup.h1.string
print(repr(text))
print(type(text))
print(type(str(text)))
```

Output Targets: HTML line 15

```
'Page & Quill Books'
<class 'bs4.element.NavigableString'>
<class 'str'>
```

Rule

Call `str()` on a NavigableString before you store it. A NavigableString keeps a reference to the whole tree. This wastes memory.

4.3 BeautifulSoup

```
print(type(soup))
print(soup.name)
print(soup.parent)
```

Output Targets: The whole document, HTML lines 1--54

```
<class 'bs4.BeautifulSoup'>
[document]
None
```

4.4 Comment

```
from bs4 import Comment

note = soup.footer.find(string=lambda s: isinstance(s, Comment))
print(type(note))
print(repr(note))
```

Output Targets: HTML line 50

```
<class 'bs4.element.Comment'>
' Prices updated 2026-10-01 '
```

5 Navigate the tree

Navigation moves from one node to a related node. Figure 3 shows the relations for `li#b2` (lines 25–29).

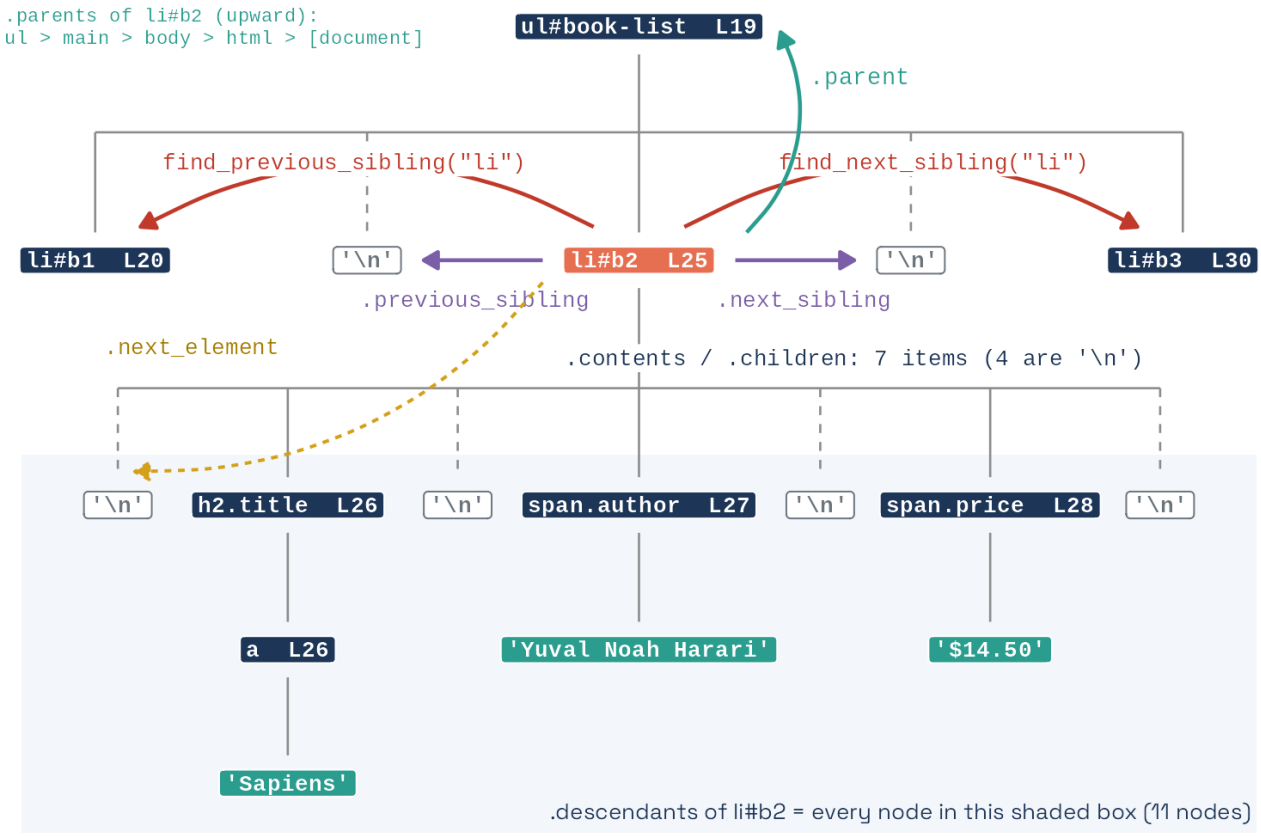


Figure 3. Navigation from `li#b2`. Dashed white boxes are whitespace strings (`'\n'`). They are real children and real siblings. Generated with R (ggplot2).

Direction	Attribute	Returns
Down	`.tagname`, `.contents`, `.children`, `.descendants`	First tag by name; list of children; iterator of children; iterator of all nodes below
Up	`.parent`, `.parents`	Containing tag; iterator up to the BeautifulSoup object
Sideways	`.next_sibling`, `.previous_sibling` (+s)	Node at the same level (often a whitespace string)
Parse order	`.next_element`, `.previous_element` (+s)	Node parsed just after / before
Text	`.string`, `.strings`, `.stripped_strings`	Single string or `None`; all strings; strings without extra whitespace

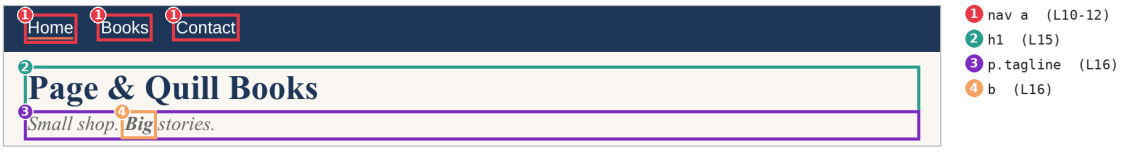


Figure 4. Targets of the examples in Sections 5.1 and 5.5: the navigation links, the `h1`, and the tagline with its `b` tag.

5.1 Go down

Use a tag name as an attribute. It returns the *first* tag with that name. It is a short form of `find()`.

```
print(soup.title)
print(soup.nav.a)
print(soup.li.h2.a)
```

Output — Targets: HTML lines 5, 10, 21

```
<title>Page &amp;amp; Quill Books</title>
<a class="nav-link active" href="/">Home</a>
<a href="/books/dune">Dune</a>
```

`.contents` is a list of the direct children. It includes the whitespace strings between tags.

```
kids = soup.nav.contents
print(len(kids))
print(kids[:3])
```

Output — Targets: HTML lines 10, 11, 12, 13

```
7
['\n', <a class="nav-link active" href="/">Home</a>, '\n']
```

`.children` is an iterator over the same children. Test `child.name` to skip strings. A string has `.name` equal to `None`.

```
for child in soup.nav.children:
    if child.name:            # skip whitespace strings
        print(child.name, child["href"])
```

Output — Targets: HTML lines 10, 11, 12

```
a /
a /books
a /contact
```

`.descendants` goes through all nodes below a tag, at every depth.

```
h2 = soup.find("h2")
print(len(h2.contents), len(list(h2.descendants)))
for d in h2.descendants:
    print(repr(d))
```

Output — Targets: HTML line 21

```
1 2
<a href="/books/dune">Dune</a>
'Dune'
```

5.2 Go up

```
price = soup.find("span", class_="price")
print(price.parent.name, price.parent["id"])
print(soup.html.parent.name)

link_text = soup.find(string="Sapiens")
print([p.name for p in link_text.parents])
```

Output — Targets: HTML lines 20--24, 26

```
li b1
[document]
['a', 'h2', 'li', 'ul', 'main', 'body', 'html', '[document]']
```

5.3 Go sideways

Siblings have the same parent. In a real page, the next sibling of a tag is usually a whitespace string. In this run, bs4 stored each whitespace run between tags as `'\n'`.

```
b1 = soup.find(id="b1")
print(repr(b1.next_sibling))
print(b1.next_sibling.next_sibling["id"])
print(repr(b1.previous_sibling))
```

Output Targets: HTML lines 20--24, 25--29

```
'\n'
b2
'\n'
```

```
b3 = soup.find(id="b3")
print([s["id"] for s in b3.next_siblings if s.name])
print([s["id"] for s in b3.previous_siblings if s.name])
```

Output Targets: HTML lines 20--24, 25--29, 30--34, 35--39

```
['b4']
['b2', 'b1']
```

5.4 Go back and forth (parse order)

`.next_element` is the node that the parser read next. It is often a child, not a sibling.

```
link = soup.find("a", href="/books/dune")
print(repr(link.next_sibling))
print(repr(link.next_element))
print(repr(link.next_element.next_element))
print(link.previous_element.name)
```

Output Targets: HTML line 21

```
None
'Dune'
'\n'
h2
```

5.5 Get the strings

- `.string` gives the text when the tag has exactly one child string (or one child tag with one string).
- `.string` gives `None` when the tag has more than one child.
- `.strings` gives every string below the tag. `.stripped_strings` removes extra whitespace and skips whitespace-only strings.

```
print(repr(soup.h1.string))
print(repr(soup.find("h2").string))     # one child chain
print(soup.find(class_="tagline").string)  # mixed content
```

Output Targets: HTML lines 15, 16, 21

```
'Page & Quill Books'
'Dune'
None
```

```
tagline = soup.find(class_="tagline")
print(list(tagline.strings))
print(list(soup.find(id="b2").stripped_strings))
```

Output Targets: HTML lines 16, 25--29

```
['Small shop. ', 'Big', ' stories.']
['Sapiens', 'Yuval Noah Harari', '$14.50']
```

6 Search the tree

6.1 find() and find_all()

Method	Returns	If nothing matches
`find(name, attrs, recursive, string, **kwargs)`	The first match	`None`
`find_all(name, attrs, recursive, string, limit, **kwargs)`	A `ResultSet` (a list) of all matches	An empty list `[]`

```
print(soup.find("h2"))
print(len(soup.find_all("h2")))
print(soup.find("aside"))
print(soup.find_all("aside"))
```

Output — Targets: HTML lines 21, 26, 31, 36

```
<h2 class="title"><a href="/books/dune">Dune</a></h2>
4
None
[]
```

6.2 Kinds of filter

You can give each argument (`name`, an attribute, `string`) one of five filter kinds.

Filter	Example	Matches
String	`"h2"`	An exact match
Regular expression	`re.compile("^h")`	A match of `re.search()`
List	`["h1", "h2"]`	A match of any item
`True`	`href=True`	Any tag that has the attribute (or any tag, for `name`)
Function	`def f(tag): ...`	A tag for which the function returns `True`



Figure 5. Targets of the class, id, attribute and CSS selector examples in the book list (lines 19–40).

By name: list, regex, True

```
for t in soup.find_all(["h1", "h2"]):
    print(t.name, t.get_text())
```

Output — Targets: HTML lines 15, 21, 26, 31, 36

```
h1 Page & Quill Books
h2 Dune
h2 Sapiens
h2 The Hobbit
h2 Atomic Habits
```

```
names = [t.name for t in soup.find_all(re.compile("^h"))]
print(names)
```

Output — Targets: HTML lines 2--54

```
['html', 'head', 'header', 'h1', 'h2', 'h2', 'h2', 'h2']
```

The regex `"^h"` matches every name that starts with `h`: `html`, `head`, `header`, `h1`, `h2`.

```
print(len(soup.find_all(True)))
print(len(soup.find_all(href=True)))
```

Output — Targets: HTML lines 6, 10, 11, 12, 21, …

```
55
9
```

By CSS class: class_

`class` is a reserved word in Python. Use `class_` (with an underscore). A tag matches if any one of its classes matches. A string with two classes must match the attribute value exactly, in the same order.

```
prices = soup.find_all("span", class_="price")
print([p.get_text() for p in prices])
print(soup.find_all(class_="sale"))
print(soup.find_all(class_="price sale"))
print(soup.find_all(class_="sale price"))
```

Output — Targets: HTML lines 23, 28, 33, 38

```
['$9.99', '$14.50', '$7.25', '$11.00']
[<span class="price sale">$11.00</span>]
[<span class="price sale">$11.00</span>]
[]
```

By id and other attributes

Each unknown keyword argument filters on the attribute with that name. Some names (for example `data-genre`) are not valid Python keywords. Put them in the `attrs` dictionary.

```
print(soup.find(id="b3").h2.get_text())
print(soup.find("li", id="b9"))
fiction = soup.find_all("li", attrs={"data-genre": "fiction"})
print([li["id"] for li in fiction])
print(soup.find("meta", attrs={"charset": True}))
```

Output — Targets: HTML lines 4, 20--24, 30--34

```
The Hobbit
None
['b1', 'b3']
<meta charset="utf-8"/>
```

```
book_links = soup.find_all("a", href=re.compile(r"^/books/"))
print([a["href"] for a in book_links])
```

Output — Targets: HTML lines 21, 26, 31, 36

```
['/books/dune', '/books/sapiens', '/books/hobbit', '/books/atomic-habits']
```

By text: string

`string` searches strings, not tags. With a tag name, it finds tags whose `.string` matches.

```
print(soup.find_all(string=re.compile(r"^\$")))
print(soup.find_all("a", string="Books"))
print(soup.find_all(string=["Dune", "Sapiens"]))
```

Output — Targets: HTML lines 23, 28, 33, 38

```
['$9.99', '$14.50', '$7.25', '$11.00']
[<a class="nav-link" href="/books">Books</a>]
['Dune', 'Sapiens', 'Dune', 'Sapiens']
```

With a function

```
def cheap_book(tag):
    if tag.name != "li":
        return False
    price = tag.find("span", class_="price").get_text()
    return float(price.lstrip("$")) < 10

print([li["id"] for li in soup.find_all(cheap_book)])
```

Output — Targets: HTML lines 20--24, 30--34

```
['b1', 'b3']
```

limit and recursive

`limit` stops the search after *n* matches. `recursive=False` examines only the direct children. Only `find()` and `find_all()` accept `recursive`.

```
print(soup.find_all("a", limit=2))
```

Output — Targets: HTML lines 10, 11

```
[<a class="nav-link active" href="/">Home</a>, <a class="nav-link" href="/books">Books</a>]
```

```
ul = soup.find("ul")
print(len(ul.find_all("li", recursive=False)))
print(ul.find_all("a", recursive=False))
print(len(ul.find_all("a")))
```

Output — Targets: HTML lines 19--40

```
4
[]
4
```

Calling a tag is the same as calling `find_all()` on it:

```
print(len(soup("a")))          # same as soup.find_all("a")
print(soup.footer("a"))
```

Output — Targets: HTML line 51

```
8
[<a href="mailto:hello@pagequill.example">Email us</a>]
```

6.3 Search in other directions

These methods take the same filters. Each plural method returns a list. Each singular method returns the first match or `None`.

All matches	First match	Direction (iterator used)
`find_parents()`	`find_parent()`	Up (`.parents`)
`find_next_siblings()`	`find_next_sibling()`	Later siblings (`.next_siblings`)
`find_previous_siblings()`	`find_previous_sibling()`	Earlier siblings (`.previous_siblings`)
`find_all_next()`	`find_next()`	Everything after, in parse order (`.next_elements`)
`find_all_previous()`	`find_previous()`	Everything before, in parse order (`.previous_elements`)

```
price = soup.find(string="$14.50")
print(price.find_parent("li")["id"])
print([p.name for p in price.find_parents(["li", "ul", "main"])])
```

Output — Targets: HTML line 28

```
b2
['li', 'ul', 'main']
```

```
author = soup.find(id="b4").find(class_="author")
print(author.find_next_sibling("span"))
print(author.find_previous_sibling("h2").a["href"])
print([li["id"] for li in soup.find(id="b1").find_next_siblings("li", limit=2)])
```

Output — Targets: HTML lines 25--29, 30--34, 36, 37, 38

```
<span class="price sale">$11.00</span>
/books/atomic-habits
['b2', 'b3']
```

```
print(soup.h1.find_next("a"))
print(soup.find("table").find_previous("span"))
print(soup.find("th").find_all_next("td", limit=2))
```

Output — Targets: HTML lines 21, 38, 43

```
<a href="/books/dune">Dune</a>
<span class="price sale">$11.00</span>
[<td>Dune</td>, <td>12</td>]
```

7 CSS selectors: select() and select_one()

`select()` returns a list of all matches. `select_one()` returns the first match or `None`. The Soup Sieve package runs the selectors. You can also call them as `soup.css.select()`.

Selector	Meaning	Example on the page
`tag` / `.cls` / `#id`	Name / class / id	`li`, `.price`, `#b2`
`.a.b`	Has both classes	`span.price.sale`
`A B`	B anywhere inside A	`#b2 .price`
`A > B`	B is a direct child of A	`li.book > h2`
`A + B` / `A ~ B`	B is the next sibling / any later sibling of A	`#topnav a + a`
`[attr]` / `[attr="v"]`	Has the attribute / exact value	`a[href]`
`[attr^="v"]`, `[attr$="v"]`, `[attr*="v"]`	Starts with / ends with / contains	`a[href^="mailto:"]`
`:nth-of-type(n)`	The n-th sibling of that type	`td:nth-of-type(2)`
`A, B`	A or B	`h1, h2`

```
titles = soup.select("li.book > h2 a")
print([a.get_text() for a in titles])
print(soup.select_one("#b2 .price").get_text())
```

Output — Targets: HTML lines 21, 26, 31, 36

```
['Dune', 'Sapiens', 'The Hobbit', 'Atomic Habits']
$14.50
```

```
print(soup.select("li.bestseller")[0]["id"])
print(soup.select("span.price.sale"))
print(soup.select("nav a.active"))
```

Output — Targets: HTML lines 10, 25--29, 38

```
b2
[<span class="price sale">$11.00</span>]
[<a class="nav-link active" href="/">Home</a>]
```

```
print([a["href"] for a in soup.select('a[href^="/books/"]')][:2])
print([t.get_text() for t in soup.select('li[data-genre="fiction"] .title')])
print(soup.select('a[href^="mailto:"]')[0].get_text())
```

Output — Targets: HTML lines 21, 31, 51

```
['/books/dune', '/books/sapiens']
['Dune', 'The Hobbit']
Email us
```

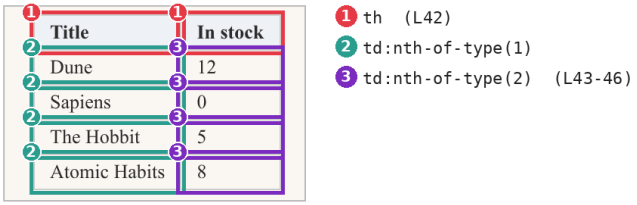


Figure 6. Targets of `#stock tr td:nth-of-type(2)` (lines 43–46) and of the table extraction in Section 8.

```
stock = soup.select("#stock tr td:nth-of-type(2)")
print([td.get_text() for td in stock])
print(soup.select("#topnav a + a")[0].get_text())
print(soup.select("#b1 ~ li")[-1]["id"])
```

Output — Targets: HTML lines 43, 44, 45, 46

```
['12', '0', '5', '8']
Books
b4
```

`select()` on a tag searches only inside that tag. With no match, `select()` gives `[]` and `select_one()` gives `None`:

```
b2 = soup.find(id="b2")
print([s.get_text() for s in b2.select("span")])
print(soup.select(".bargain"), soup.select_one(".bargain"))
```

Output — Targets: HTML lines 27, 28

```
['Yuval Noah Harari', '$14.50']
[] None
```

8 Get the data

Code	Result
`tag.get_text()` / `tag.text`	All text below the tag, joined into one string
`tag.get_text(" | ", strip=True)`	The pieces stripped and joined with the separator
`tag["href"]`	The attribute value. A `KeyError` if it is missing
`tag.get("href")`	The value, or `None` (or your default) if it is missing
`tag.attrs`	A dictionary of all attributes

```
tagline = soup.find(class_="tagline")
print(repr(tagline.get_text()))
print(repr(tagline.text))
print(soup.find(id="b2").get_text(" | ", strip=True))
```

Output — Targets: HTML lines 16, 25--29

```
'Small shop. Big stories.'
'Small shop. Big stories.'
Sapiens | Yuval Noah Harari | $14.50
```

In this run, `get_text()` skipped the comment on line 50:

```
print(repr(soup.footer.get_text(strip=True)))
```

Output — Targets: HTML lines 49--52

```
'© 2026 Page & Quill.Email us'
```

```
link = soup.select_one("#b3 h2 a")
print(link["href"])
print(link.get("title"))
print(link.get("title", "no title"))
print(link["title"])
```

Output (raises an error) — Targets: HTML line 31

```
/books/hobbit
None
no title
KeyError: 'title'
```

```
print(soup.find(id="b2").attrs)
print(soup.find("link").attrs)
```

Output — Targets: HTML lines 6, 25--29

```
{'class': ['book', 'bestseller'], 'id': 'b2', 'data-genre': 'history'}
{'rel': ['stylesheet'], 'href': 'style.css'}
```

Read the stock table into a list of rows:

```
rows = []
for tr in soup.select("#stock tr"):
    cells = [c.get_text(strip=True) for c in tr.find_all(["th", "td"])]
    rows.append(cells)
print(rows)
```

Output — Targets: HTML lines 42, 43, 44, 45, 46

```
[['Title', 'In stock'], ['Dune', '12'], ['Sapiens', '0'], ['The Hobbit', '5'], ['Atomic Habits',
↪  '8']]
```

9 Modify the tree (brief)

Edits change the tree in memory. They do not change the file. Each example below starts from a fresh soup.

Method	Action
`tag["attr"] = v`, `del tag["attr"]`	Set or remove an attribute
`tag.string = "text"`	Replace all contents with one string
`soup.new_tag(name, attrs={...})`, `tag.append(x)`	Make a tag; add a child at the end
`x.replace_with(y)`	Put `y` in the place of `x`; return `x`
`x.extract()`	Remove `x`; return it
`tag.decompose()`	Remove the tag and destroy it

```
li = soup.find(id="b3")
li["data-genre"] = "fantasy"
li["class"].append("sale")
del li["id"]
print(li.attrs)
```

Output Targets: HTML lines 30--34

```
{'class': ['book', 'sale'], 'data-genre': 'fantasy'}
```

```
new = soup.new_tag("li", attrs={"class": "book", "id": "b5"})
new.string = "Coming soon"
soup.find("ul").append(new)
print(soup.find("ul").find_all("li", recursive=False)[-1])
print(len(soup.select("li.book")))
```

Output Targets: Adds a child after HTML line 39 (inside the ul)

```
<li class="book" id="b5">Coming soon</li>
5
```

```
soup.find("nav").decompose()
print(soup.find("nav"))
print(soup.body.find(True).name)
```

Output Targets: Removes HTML lines 9--13

```
None
header
```

```
old = soup.find(string="$7.25")
old.replace_with("$6.00")
print(soup.find(id="b3").find(class_="price"))
```

Output Targets: HTML line 33

```
<span class="price">$6.00</span>
```

```
soup.h1.string = "Page & Quill: Autumn Sale"
print(soup.h1)
```

Output Targets: HTML line 15

```
<h1>Page &amp;amp; Quill: Autumn Sale</h1>
```

```
note = soup.find(string=lambda s: isinstance(s, Comment))
removed = note.extract()
print(repr(removed))
print(soup.footer.find(string=lambda s: isinstance(s, Comment)))
```

Output Targets: HTML line 50

```
' Prices updated 2026-10-01 '
None
```

10 Output

`prettify()` gives an indented string with one tag or string per line. Use it to read the structure. Do not use it to rewrite a page, because it adds whitespace. `str()` gives the HTML with no added formatting. `encode()` gives UTF-8 bytes.

```
print(soup.find("h2").prettify())
```

Output — Targets: HTML line 21

```
<h2 class="title">
 <a href="/books/dune">
  Dune
 </a>
</h2>
```

```
print(str(soup.find(class_="tagline")))
print(soup.footer.p.encode()[:12])
```

Output — Targets: HTML lines 16, 51

```
<p class="tagline">Small shop. <b>Big</b> stories.</p>
b'<p>\xc2\xa9 2026 P'
```

11 A short scraping workflow

This workflow downloads the page with `requests`, extracts the books, and saves a CSV file. In this guide, a local test server serves `bookstore.html`:

```
cd folder-with-bookstore-html
python -m http.server 8765
```

Output (the server keeps running) — Targets: Serves the whole file

```
Serving HTTP on 0.0.0.0 port 8765 (http://0.0.0.0:8765/) ...
```

1. Check that the site allows scraping (its terms and `robots.txt`).
2. Download the page. Set a timeout. Send a clear User-Agent.
3. Stop on HTTP errors with `raise_for_status()`.
4. Parse `resp.content` (bytes). Then bs4 detects the encoding.
5. Select one container per record (`li.book`).
6. Read each field inside the container. Convert the types.
7. Make absolute URLs with `urljoin()`.
8. Load the records into pandas. Save the result.

```
import requests

URL = "http://localhost:8765/bookstore.html"
headers = {"User-Agent": "krish-learning-scraper/0.1"}
resp = requests.get(URL, headers=headers, timeout=10)
resp.raise_for_status()
print(resp.status_code, resp.headers["Content-Type"])
print(len(resp.content), "bytes")
```

Output — Targets: The whole page, served by a local test server

```
200 text/html
1868 bytes
```

```python
import requests
from urllib.parse import urljoin

URL = "http://localhost:8765/bookstore.html"
resp = requests.get(URL, timeout=10)
soup = BeautifulSoup(resp.content, "html.parser")
print(soup.original_encoding)

books = []
for li in soup.select("li.book"):
    link = li.select_one("h2 a")
    books.append({
        "id": li["id"],
        "title": link.get_text(strip=True),
        "author": li.select_one(".author").get_text(strip=True),
        "price": float(li.select_one(".price").get_text().lstrip("$")),
        "genre": li.get("data-genre"),
        "url": urljoin(URL, link["href"]),
    })
print(books[0])
```

Output Targets: HTML lines 20--24, 25--29, 30--34, 35--39

```
utf-8
{'id': 'b1', 'title': 'Dune', 'author': 'Frank Herbert', 'price': 9.99, 'genre': 'fiction', 'url':
↪  'http://localhost:8765/books/dune'}
```

```python
import requests
import pandas as pd

resp = requests.get("http://localhost:8765/bookstore.html", timeout=10)
soup = BeautifulSoup(resp.content, "html.parser")

rows = [{"title": li.h2.get_text(strip=True),
         "price": float(li.select_one(".price").get_text()[1:])}
        for li in soup.select("li.book")]
df = pd.DataFrame(rows)

stock = {}
for tr in soup.select("#stock tr"):
    cells = tr.find_all("td")
    if cells:                          # the header row has th, not td
        stock[cells[0].get_text()] = int(cells[1].get_text())
df["stock"] = df["title"].map(stock)

print(df)
df.to_csv("/tmp/books.csv", index=False)
```

Output Targets: HTML lines 20--24, 25--29, 30--34, 35--39, 42, ...

```
           title  price  stock
0           Dune   9.99     12
1        Sapiens  14.50      0
2     The Hobbit   7.25      5
3  Atomic Habits  11.00      8
```

Good practice

- Wait between requests (for example `time.sleep(1)`). Do not overload a server.
- Select by stable hooks (`id`, `data-*`, meaningful classes). Do not select by position only.
- Save the raw HTML once. Then develop the parser offline against the saved file.
- bs4 does not run JavaScript. If the data is missing from `resp.content`, the page builds it in the browser.

12 Common pitfalls

Symptom	Cause	Fix
`AttributeError: 'NoneType' ...`	`find()` found nothing and returned `None`	Test the result before you use it
`AttributeError: ResultSet ...`	You used a list from `find_all()` as one tag	Loop over the list, or use `find()`
`SyntaxError` with `class=`	`class` is a Python keyword	Use `class_=` or `attrs={"class": ...}`
`.string` is `None`	The tag has more than one child	Use `get_text()`
Sibling is `'\n'`	Whitespace between tags is a string	Use `find_next_sibling("tag")`
Wrong characters (Â©)	`resp.text` was decoded with a wrong guess	Pass `resp.content` (bytes) to bs4
Selector from DevTools finds nothing	The browser adds `<tbody>`; some parsers do not	Remove `tbody` from the selector
`KeyError: 'class'`	The attribute is missing	Use `tag.get("class")`

`.string` is `None` for mixed content: see the `tagline` example in Section 5.5. Whitespace siblings: see Section 5.3.

find() returns None

```
print(soup.find("aside").get_text())
```

Output (raises an error) — Targets: No aside element on the page

```
AttributeError: 'NoneType' object has no attribute 'get_text'
```

```
box = soup.find("aside")
text = box.get_text(strip=True) if box else "missing"
print(text)
```

Output — Targets: No aside element on the page

```
missing
```

A ResultSet is not a Tag

```
print(soup.find_all("li").find("a"))
```

Output (raises an error) — Targets: HTML lines 20--24, 25--29, 30--34, 35--39

```
AttributeError: ResultSet object has no attribute "find". You're probably treating a list of elements
↪  like a single element. Did you call find_all() when you meant to call find()?
```

class needs an underscore

```
soup.find_all("span", class="price")
```

Output (raises an error) — Targets: Syntax error: the code never runs

```
SyntaxError: invalid syntax (<p_class>, line 1)
```

Use `class_="price"` instead (Section 6.2).

Encoding: use bytes

The test server sends `Content-Type: text/html` with no charset. `requests` then guesses ISO-8859-1. The page is UTF-8, so `resp.text` is wrong. bs4 reads the bytes and the `<meta charset>` tag, and gets it right.

```
import requests

resp = requests.get("http://localhost:8765/bookstore.html", timeout=10)
print(resp.encoding)
from_text = BeautifulSoup(resp.text, "html.parser")
from_bytes = BeautifulSoup(resp.content, "html.parser")
print(from_text.footer.p.get_text()[:6])
print(from_bytes.footer.p.get_text()[:6])
```

Output Targets: HTML line 51

```
ISO-8859-1
Â© 202
© 2026
```

tbody and parser differences

A browser inserts `<tbody>` into every table. A selector copied from the browser DevTools can therefore contain `tbody`. The page source has no `tbody`.

```
sel = "#stock > tbody > tr"
for parser in ["html.parser", "lxml", "html5lib"]:
    with open("bookstore.html", encoding="utf-8") as f:
        s = BeautifulSoup(f, parser)
    print(parser, len(s.select(sel)))
```

Output Targets: HTML lines 41--47

```
html.parser 0
lxml 0
html5lib 5
```

Missing attribute: `tag["title"]` raises `KeyError` (see the last line of the `link` example in Section 8). Use `tag.get("title")`.

13 Cheat sheet

Each code cell ran on `bookstore.html` with `html.parser`. `HTML` is the page source as a string. The last group changes the tree, so it runs in order.

Task	Code	Result on bookstore.html
Parse		
Make the soup	`type(BeautifulSoup(HTML, "html.parser"))`	`<class 'bs4.BeautifulSoup'>`
Show the tree	`soup.prettify().splitlines()[1]`	`'<html lang="en">'`
Navigate		
First tag by name	`soup.h1`	`<h1>Page & Quill Books</h1>`
Direct children (list)	`len(soup.nav.contents)`	`7`
Child tags only	`[c.name for c in soup.nav.children if c.name]`	`['a', 'a', 'a']`
Parent	`soup.find("span").parent["id"]`	`'b1'`
Next tag sibling	`soup.find(id="b1").find_next_sibling("li")["id"]`	`'b2'`
Only string child	`soup.h1.string`	`'Page & Quill Books'`
All text pieces	`list(soup.find(id="b1").stripped_strings)`	`['Dune', 'Frank Herbert', '$9.99']`
Search		
First match (or None)	`soup.find("h2").get_text()`	`'Dune'`
All matches (list)	`len(soup.find_all("h2"))`	`4`
By CSS class	`len(soup.find_all(class_="price"))`	`4`
By id	`soup.find(id="b3").h2.a.string`	`'The Hobbit'`
By other attribute	`len(soup.find_all(attrs={"data-genre": "fiction"}))`	`2`
By regex on attribute	`len(soup.find_all(href=re.compile("^/books/")))`	`4`
By text	`soup.find_all(string=re.compile(r"^\$"))[:2]`	`['$9.99', '$14.50']`
Stop after n	`len(soup.find_all("a", limit=2))`	`2`
Direct children only	`len(soup.ul.find_all("li", recursive=False))`	`4`
Nearest ancestor	`soup.find(string="$14.50").find_parent("li")["id"]`	`'b2'`
CSS selectors		
All by selector	`len(soup.select("li.book > h2 a"))`	`4`
First by selector	`soup.select_one("#b2 .price").get_text()`	`'$14.50'`
Attribute prefix	`soup.select_one('a[href^="mailto:"]').string`	`'Email us'`
Extract		
Text	`soup.find(class_="tagline").get_text(" ", strip=True)`	`'Small shop. Big stories.'`
Attribute (must exist)	`soup.select_one("h2 a")["href"]`	`'/books/dune'`
Attribute (may be missing)	`soup.select_one("h2 a").get("title", "n/a")`	`'n/a'`
All attributes	`soup.find("link").attrs`	`{'rel': ['stylesheet'], 'href': 'style.css'}`
Output		
Plain HTML string	`str(soup.b)`	`'<b>Big</b>'`
Modify (run in order)		
Replace a string	`soup.b.string.replace_with("Huge")`	`'Big'`
Check the change	`str(soup.b)`	`'<b>Huge</b>'`
Remove a tag	`soup.nav.decompose()`	`None`

A Glossary

Term	Meaning
Attribute	A name–value pair in a start tag, for example `href="/books"`. bs4 gives it with `tag["href"]`.
CSS selector	A pattern that selects elements, for example `li.book > h2 a`. bs4 runs it with `select()`.
Comment	Text inside `<!-- ... -->`. bs4 gives it as a `Comment` object.
DOM	Document Object Model. The tree of elements that a browser builds from a page.
Descendant	Any node below a node: children, their children, and so on.
Element	One part of a page, from its start tag to its end tag.
Encoding	The rule that turns bytes into characters, for example UTF-8.
Entity	A code for a character in HTML, for example `&` for `&`.
HTML	HyperText Markup Language. The markup of web pages.
NavigableString	bs4's object for a piece of text in the tree.
Node	One item in the tree: a tag, a string, or a comment.
Parse tree	The tree of Python objects that bs4 builds from the markup.
Parser	A program that reads markup text and builds a tree (`html.parser`, `lxml`, `html5lib`).
ResultSet	The list that `find_all()` returns.
Sibling	A node with the same parent as another node.
Soup	The `BeautifulSoup` object: the root of the parse tree.
Tag	bs4's object for one HTML element, with a name and attributes.
Web scraping	Automatic download and extraction of data from web pages.
Whitespace string	A string of only spaces and newlines between tags.

Reference

Richardson, L. *Beautiful Soup Documentation*. https://www.crummy.com/software/BeautifulSoup/bs4/doc/ (read 12 October 2026). Soup Sieve documentation: https://facelessuser.github.io/soupsieve/.